

How MTA evaluates an app, end to end

MTA takes a GitHub repository, its Business Requirements Document (BRD) and, when the app is deployed, the live URL. It proves each acceptance criterion with code evidence and real browser tests, then produces a compliance score, gates and a report.

18

pipeline steps

6

phases, two run in parallel

5

critical steps (a failure stops the run)

E1–E5

evidence strength ladder

1

What goes in

GitHub repository

Fetches and pins to one commit. Code is indexed into a knowledge graph (routes, pages, forms, agents, prompts, tests).

BRD

Taken from the repository (a given path or auto-detected PDF, DOCX or MD), or written by MTA from the live app or the code when there is none.

Live URL + test account

Optional. MTA logs in with the project's test account and drives the real app in Chromium. The credentials are encrypted and handed only to the browser steps.

Mode	What runs
brd_and_live	Everything: code evidence plus runtime tests on the live app.
live_only	MTA writes the BRD from the authenticated runtime profile, then runs everything. It is scored as "inferred conformance".
brd_only	Code review only. No runtime tests are generated; criteria are judged from the code (at most E3).

2 System architecture

Web app

Next.js 16 with Prisma.
Creates projects and runs, shows live progress, logs and reports, and lets users cancel a run.

Postgres

Runs, steps, events, requirements, criteria, evidence, test cases and findings. The schema is managed with Prisma migrations.

Evaluation service

FastAPI (`api/server.py`) with the DAG pipeline (`api/pipeline.py`). Runs 2 evaluations at a time, sends a heartbeat every 30 s and queues runs for free worker slots.

External

Gemini (rate-limited, cached, with timeouts), Chromium via Playwright (sandboxed, SSRF-guarded), GitHub API.

3 The pipeline: a DAG, not a straight line

A step starts as soon as the steps it needs are done. The code-evidence branch (phase E) runs while the browser branch (phase D) tests the live app, which is where most of the time goes. **critical** a failure stops the run. **LLM** uses Gemini. Every other failure is recorded and listed in the report; the run continues.

A · Understand the repository starts immediately

repo_fetch **critical**

Resolves the branch and commit, then downloads the files that matter (source, docs, manifests) at that commit.

Output: repo files, commit SHA, coverage report (what was read and what was skipped)

repo_intelligence **critical**

Deterministic knowledge graph and code chunks: routes, pages, forms, agents, prompts and tests in Python, JS/TS and other languages. Cached per commit.

needs: repo_fetch

repo_analysis **LLM**

App profile: purpose, capabilities, interface type (form app, document generator, chatbot, API...) and endpoints.

needs: repo_fetch · if Gemini fails, a simpler profile is built from the code

B · See the live app browser

runtime_discovery

Logs in with the test account, then crawls up to 12 pages (40 visits, 10 min). On each page it waits for the content to render, then records forms with every field label, buttons, downloads, chat boxes and API calls. Pages the code says hold a form get a second, longer look. Item pages such as `/runs/<id>` count once.

Output: runtime profile, plus the logged-in session, which the executor reuses
needs: repo_intelligence

app_classification

Runtime plus code signals give the app type: Chatbot, Form Application, Document Generator, Dashboard, REST API, Workflow System, Static Website or Hybrid. Each type gets its own testing strategy.

needs: runtime_discovery, repo_intelligence

C · Requirements and tests

brd_builder

critical

LLM

Finds and reads the BRD (scanned PDFs are transcribed by Gemini) and extracts up to 40 requirements with atomic acceptance criteria (AC-xx.y).

- Every requirement is traced to a verified quote from the BRD.
- Each criterion gets a verifiability tag: runtime, static or both.
- Criteria the BRD text doesn't support are flagged as ungrounded.
- Anything MTA had to cut is listed under limitations.

needs: repo_analysis, runtime_discovery

agent_test_generation

LLM

Writes runtime tests only for criteria the discovered app can actually exercise, in its real interaction model (form values, chat messages, exports).

- Sized to the execution budget, e.g. 20 tests for a document generator.
- Non-observable claims (performance, infrastructure) go to code review instead.
- When nothing can run the tests (code-only mode, failed login, pages that never loaded), it generates **no tests** and says why.

needs: brd_builder, app_classification

D · Test the live app runs in parallel with phase E

`action_generation` LLM

Turns each test into grounded browser or HTTP steps that target only elements discovery actually saw. Gemini proposes the steps and the engine checks them, then completes each plan: login, every required field, submit, wait, download, capture and judged assertion.

`playwright_executor`

Runs every plan in Chromium in a fresh context, reusing discovery's session.

- Adaptive parallelism: a document generator is timed on one generation first, tests then run up to 2 at a time, and MTA drops back to one at a time if the app slows down.
- Timing tests always run alone.
- Each test has a time limit, the whole step has a budget, and a run can be cancelled at any point.

Output: step results, screenshots, page text, replies, downloaded files, network and console errors

`evidence_collection` → `output_validation` LLM → `pass_fail`

One evidence bundle per test, then deterministic checks: document produced, readable, right format, reflects the input, no server errors, within time limits. A Gemini judge then scores each pass criterion and must quote the actual output. The result is one verdict per test and per criterion (supported, refuted or unverified).

`live_agent_execution` LLM

Fallback for chat and API apps only. It retries tests the browser couldn't reach and never overwrites a result already recorded.

needs: `pass_fail`

E · Code evidence runs in parallel with phase D

`requirement_traceability` critical LLM

For each criterion:

- Hybrid retrieval (keyword search plus the code graph) finds the relevant code.
- A verifier sample judges it; a second sample runs only when the first isn't decisive.
- Citations are checked against the actual files, and MTA proves whether the code is wired to an entry point.
- If the LLM fails for a criterion, it is marked `llm_unavailable` rather than scored 0.

needs: `brd_builder`, `repo_intelligence`

repository_review

LLM

Scanner rules (secrets, CORS, debug mode, missing auth, LLM calls without retries...) plus an LLM design review. Findings MTA can't place in the code don't count against the app.

F · Verdict

brd_compliance

Merges the runtime results into the code trace (runtime evidence outranks code evidence) and builds the matrix: requirement → criterion → evidence → verdict. It also flags code-vs-runtime disagreements, such as implemented in code but broken at runtime.

needs: requirement_traceability, live_agent_execution

compliance_scoring

critical

→

final_report

Compliance score with a 90% confidence band, verification depth, scorecard, gates and recommendations, then the final report as JSON, Markdown and printable HTML.

4 How every test ends: always with a reason

Verdict	Meaning	Counted?
Passed	Deterministic checks passed and the judge found the expectation met, citing quotes from the output.	For the app
Failed	App defect: no document, wrong format, an error, a timeout, or output that doesn't meet the expectation.	Against the app
Inconclusive	An MTA limitation: it couldn't drive the UI, its browser crashed, the app rejected MTA's test data, or the output couldn't be judged.	Not counted
Not executed	Never started: blocked by login, out of the time budget, cancelled, or no matching element. The specific cause is always stored.	Not counted

5 Evidence ladder

E5

Runtime + deterministic check (credit ×1.0) e.g. the DOCX was downloaded and contains the required sections

E4

Runtime, judged with verified quotes (×0.9)

E3**Code wired to an entry point** (×0.75) cited code reachable from a route, page or form**E2****Code only** (×0.55) cited, but not proven to be reachable**E1****Unverified** (no credit)

6 Scoring and decision

Compliance = $100 \cdot \sum p(R) \cdot \text{score}(R) / \sum p(R)$ $p = 3 / 2 / 1$ for high / medium / low priority
 $\text{score}(R) = \sum w(c) \cdot \text{value}(c) \cdot \text{strength}(c) / \sum w(c)$ ungrounded criteria weigh 0.5;
 LLM-unavailable criteria are excluded

Gates

- **Block:** a critical security finding, a failed high-priority requirement, or more than 25% of criteria the LLM couldn't check.
- **Warn:** login failed, the app is behind a login, nothing executed, many tests inconclusive, broken at runtime, low compliance, shallow verification, insufficient evidence.

Outcomes

- Compliant / Compliant with warnings (75 or more)
- Partially compliant (50–75) · Not compliant (below 50, or blocked)
- **Not verified at runtime:** the app is deployed but no criterion was proven on it
- Not assessable: nothing scorable

7 Run lifecycle and reliability

queued

Waits for a free worker slot (2 at a time).

→

running

Heartbeat every 30 s. The run view shows live steps, logs and an estimate of the time left.

→

completed · failed · cancelled

Any test still pending gets an explicit reason.

- **Service stopped or restarted mid-run:** the run stops at its next checkpoint, goes back to the queue and restarts automatically, once.
- **No heartbeat for 10 minutes:** the run is failed automatically. Users can cancel a run from the UI.
- **Gemini:** 180 s timeout, at most 6 concurrent calls and 120 per minute, backoff that honours Retry-After, and a 14-day response cache for re-runs.

- **Hard limits:** discovery 10 min, each test 15 min, browser execution 60–90 min. Run artifacts are pruned after 30 days.

8 Safety

SSRF guard

Live URLs must be public hosts. Private, loopback and cloud-metadata addresses are refused, including redirects and every browser request.

Prompt injection

Repo code, BRD text, page content and the app's own output are fenced as untrusted data in every prompt. A judge's "pass" must quote the real output.

Secrets

The test account is encrypted with AES-GCM and decrypted only in memory, for the browser steps. It is never logged or returned by the API. Chromium runs sandboxed.

Source: `workflows/agent_evaluation.yaml` (the DAG), `api/pipeline.py` (scheduler), `engines/runtime/*` (discovery, actions, executor, evidence), `engines/scoring/v1.py` (compliance-v1.2), `engines/report/final.py` .